Fault-Tolerant Data Processing System

You are tasked with designing and partially building a generic data processing system used by multiple internal clients.

The system ingests unreliable and inconsistent data, processes it safely, and exposes aggregated results.

NOTE: This task is **not expected to be completed**.
We care more about **judgment, structure, and trade-offs** than completeness.

You may use **any tools**, **including AI assistants.**

Problem Statement

You are building a data ingestion and processing service that receives events from multiple external clients.

These clients:

- Do not follow a strict schema
- Change formats without notice
- May resend events
- May fail mid-request

Your system must:

- Normalize incoming data
- Avoid duplicate processing
- Handle partial failures safely
- Provide consistent aggregated outputs

Functional Requirements

1. Event Ingestion

Clients send events in the following **unreliable format**:

JSON: {

```
  "source": "client_A",
  "payload": {
    "metric": "value",
    "amount": "1200",
    "timestamp": "2024/01/01"
  }
}
```

Important notes:

- Field names may differ across clients
- Types may be inconsistent (strings instead of numbers, dates in different formats)
- Some fields may be missing
- Clients may add extra fields without notice

2. Normalisation Layer

Design a **normalisation layer** that converts raw events into a canonical internal format.

Example canonical format:

```
{
  "client_id": "client_A",
  "metric": "value",
  "amount": 1200,
  "timestamp": "2024-01-01T00:00:00Z"
}
```

Your solution should:

- Handle missing or malformed fields gracefully
- Avoid breaking when new fields appear
- Clearly separate raw input from normalized data

You must decide:

- Where this logic lives
- How configurable it is
- How strict validation should be

3. Idempotency & Deduplication

Clients may resend events.

Constraints:

- No guaranteed unique event ID
- Timestamps may not be reliable
- Events may be retried after partial failures

Your system must:

- Prevent double counting
- Remain safe across retries
- Avoid relying on fragile assumptions

Full implementation is **not required**, but your design must address this clearly.

4. Partial Failure Handling

Assume the following scenario can occur:

1. Event is received
2. Event is validated
3. Database write fails
4. Client retries the request

Your system must not:

- Lose valid data
- Process the same event twice
- Enter an inconsistent state

5. Query & Aggregation API

Expose an API endpoint that:

- Returns aggregated data (e.g., totals, counts)
- Supports basic filtering (client, time range)
- Remains consistent despite retries and failures

Aggregation logic should be:

- Clearly separated from ingestion
- Designed for future extensibility

6. Frontend (Minimal)

Build a minimal UI that allows:

- Manual submission of raw events
- Toggling a “simulate failure” option
- Viewing:
    - Successfully processed events
    - Rejected or failed events
    - Aggregated results

Design polishing could get you extra points.

Constraints

Please **do not**:

- Implement authentication
- Add Docker or microservices
- Over-engineer infrastructure
- Hardcode client-specific logic everywhere

Focus on **clean structure and decision-making**.

Deliverables

By the end of 60 minutes, submit:

1. Your code (partial is completely fine)
2. A README that answers **only** the following:
    - What assumptions did you make?
    - How does your system prevent double counting?
    - What happens if the database fails mid-request?
    - What would break first at scale?

Submit your assignment on https://forms.gle/z6XCGvsXPREn7ihA6

Optional Extension (Not Required)

If time permits, consider:

“Schemas and formats may evolve over time. Old data must remain queryable.”

What We Are Evaluating

We are **not** evaluating:

- Speed
- UI polish
- Framework knowledge

We **are** evaluating:

- System thinking
- Data modeling
- Failure handling
- Ability to explain decisions